

从零理解 dimos 的建图与导航

给完全没接触过 SLAM / 自主导航的工程师看的教程。先用大白话讲清"机器人到底在做什么"，再带你一层层走进 dimos 的代码实现。配大量流程图，最后能上手用、改、扩展。

基于 dimos `main` 分支 `d2e695b38` (2026-05-13 同步) 的代码。

2026-05-14 更新说明：上游合并了 Nav Stack 0.1 (新增 `dimos/navigation/nav_stack/`)，这是一套全新的、基于 C++ NativeModule 的导航栈，目前主要给 Unitree G1 humanoid + FastLIO2 + Livox 用。Go2 仍然用本教程里描述的"老栈" (VoxelGridMapper + CostMapper + ReplanningAStarPlanner + WavefrontFrontierExplorer + PatrollingModule + MovementManager)，核心架构没变。新老两套栈的对比参见 [二、新老导航栈对比](#)。

目录

- [一、通俗篇：建图和导航是怎么回事](#)
 - [二、总览：dimos 的四层架构](#)
 - [新增：Nav Stack 0.1 — 另一条平行路线](#)
 - [三、建图 \(Mapping\) — 让机器人"看见"环境](#)
 - [四、导航 \(Navigation\) — 让机器人"会走路"](#)
 - [五、自主探索 \(Frontier Exploration\) — 让机器人"自己出门"](#)
 - [六、端到端实战 — 用 Go2 做一次完整任务](#)
 - [七、扩展点和延伸阅读](#)
 - [7.5 Nav Stack 0.1 完全指南](#)
-

一、通俗篇：建图和导航是怎么回事

1.1 机器人为什么需要"地图"？

你蒙着眼走路会撞墙。机器人也是一样：要会走路，必须先知道"哪里能走、哪里有墙、哪里有桌子"。这件事就叫**建图 (Mapping)**。

核心问题：机器人身上挂着传感器（雷达、相机、IMU），它收到的是一堆"距离"和"图像"，怎么变成一张"地图"？

1.2 SLAM 是什么

SLAM = **S**imultaneous **L**ocalization **A**nd **M**apping = "同时定位与建图"。

机器人面对一个鸡生蛋蛋生鸡的问题：

- 想建图 → 需要知道自己在哪（不然每次扫的点云就拼不起来）
- 想知道在哪 → 需要一张图作参照（不然 GPS 在室内根本不准）

SLAM 就是同时解这两个问题的算法。dimos **不自己实现 SLAM**，而是把它当作"外部黑盒"来用：

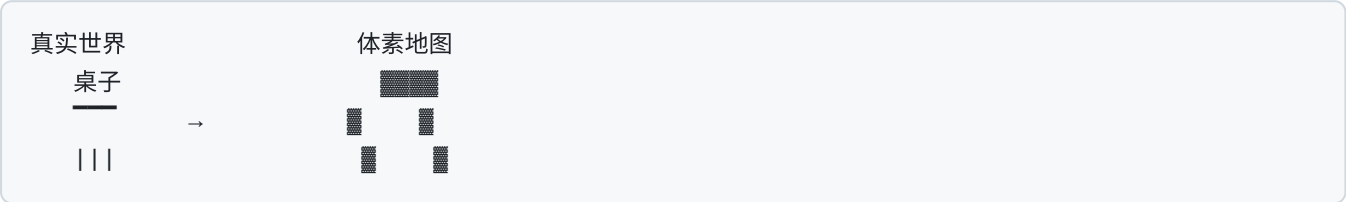
你的硬件	用什么 SLAM	提供什么输出
Livox MID-360 雷达	FastLIO2	odom （机器人在世界里的姿态）+ lidar （一帧扫描点云，已对齐到世界坐标系）
Unitree Go2 自带	Go2 SLAM	同上
仿真环境 MuJoCo	仿真器直接给	同上（甚至不需要 SLAM）

第一个关键点： odom （odometry，里程计）= 机器人在世界里的位置 + 朝向。后面所有规划都建立在这个之上。

1.3 三种地图：从粗到细

体素地图（Voxel Map） — 3D 的乐高积木

把空间切成 $5\text{cm} \times 5\text{cm} \times 5\text{cm}$ 的小方块（叫 voxel），雷达扫到哪个方块里有点，就把这个方块标记为"占据"。整个房间就被表示成了一堆乐高积木。



适合：3D 可视化、避障、表示完整 3D 结构。

占据栅格（Occupancy Grid） — 2D 的格子图

机器人通常只在地面上走，3D 太冗余。把地面切成 $5\text{cm} \times 5\text{cm}$ 的小格子，每个格子标三种状态：

值	含义	颜色
0 (FREE)	空地，能走	白
100 (OCCUPIED)	墙/障碍物	黑
-1 (UNKNOWN)	没扫到，不知道	灰

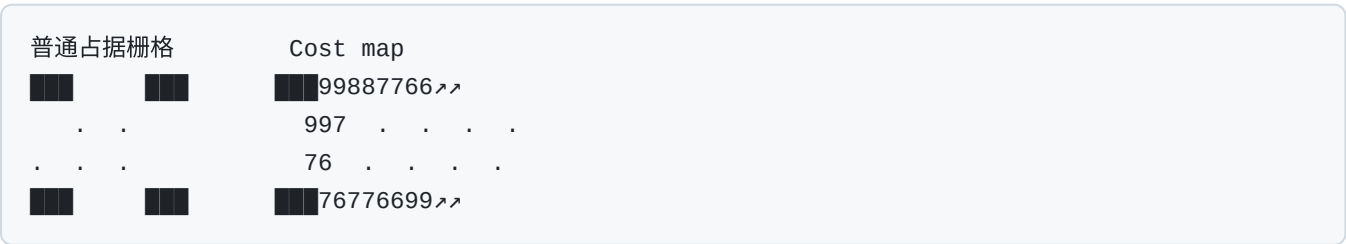
这就是经典的**占据栅格地图（OccupancyGrid）**。它比体素地图小得多，足够规划路径用。

Cost map（成本图） — 占据栅格的精装修版

只有 0/100/-1 太粗。实际规划时希望：

- 离墙远的地方便宜（cost 小）
- 离墙近的地方贵（cost 大），逼着机器人走中间
- 完全过不去的地方禁止（cost = 100）

cost map 就是这样一张"灰度图"：每个格子的值在 0-100 之间，越大越不想去。规划时找 cost 之和最小的路。



第二个关键点：dimos 的规划是在 **cost map** 上跑 **A***，不是直接在占据栅格上跑。

1.4 路径规划：A* 算法的直观理解

给定起点和终点，找一条最便宜（或最短）的路。

最经典的算法叫 **A*** (A-star)。直观地说：

1. 从起点开始，每次"看一圈"四周 8 个邻居格子
2. 给每个邻居算两个分数：- **g**：从起点到这里实际走过的成本 - **h**：从这里到终点的"直线估计"成本（叫**启发函数**）
3. 优先去 **g + h** 最小的格子继续展开
4. 反复，直到到达终点
5. 然后回溯走过来的路径

A* 比"暴力搜所有可能"快得多，因为 **h** 让它一直朝着终点方向"瞄"。

1.5 控制器：把"路径"翻译成"轮子转速"

规划器吐出来的是"经过点 $P1 \rightarrow P2 \rightarrow P3 \rightarrow \dots \rightarrow$ 终点"的一串坐标。但机器人只听得懂 **Twist**（线速度 + 角速度）的命令。

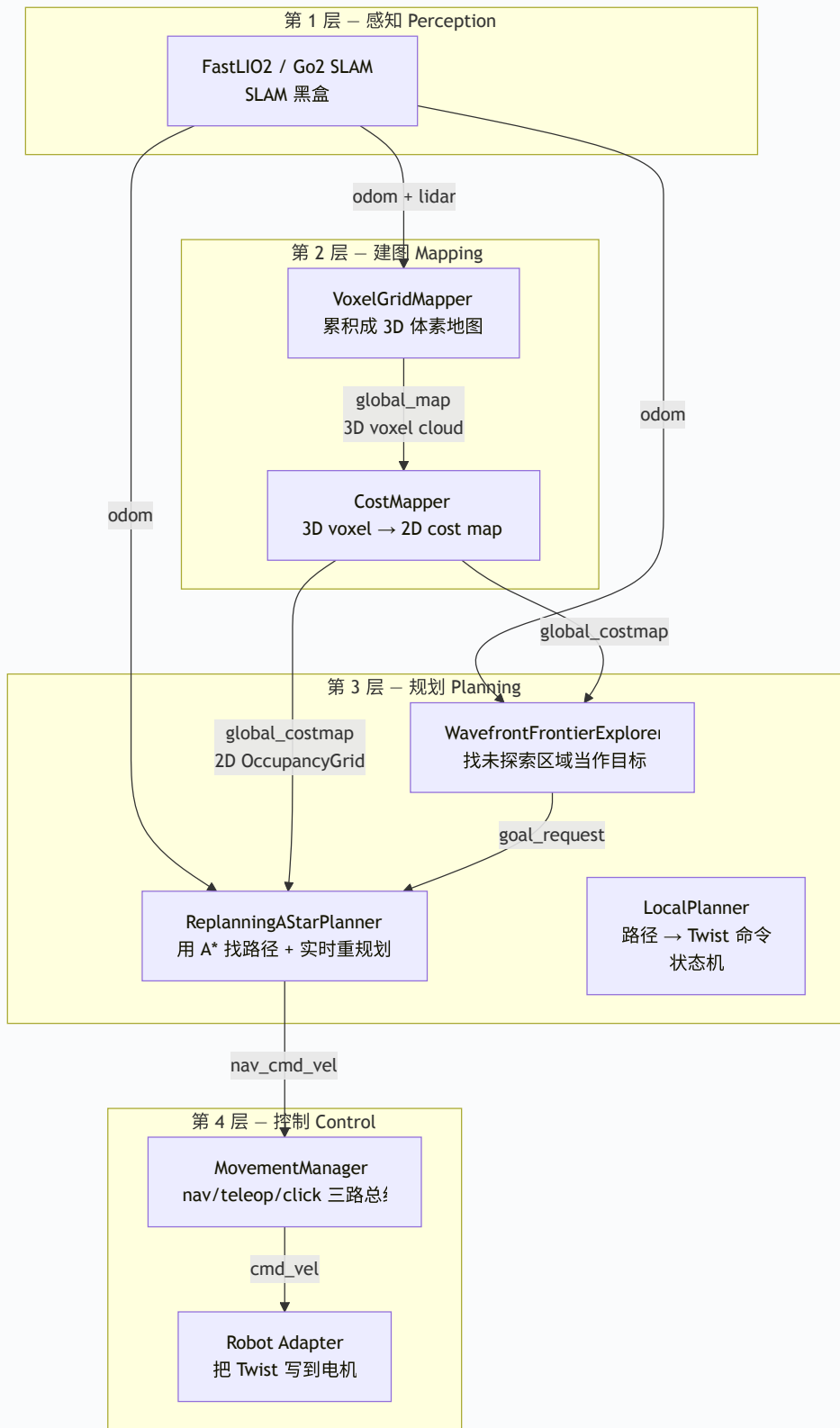
控制器 (controller) 干的就是这个翻译活：

- 看自己当前位置和朝向
- 看路径上前方一个点（叫 **lookahead 点**）
- 算出"我应该转多少度、走多快"
- 输出 `Twist(linear=v, angular= $\omega$ )`

dimos 用的是**比例控制 (P controller)**：误差越大，纠偏越用力。简单但够用。

二、总览：dimos 的四层架构

dimos 的"建图 + 导航"完整链路可以分成 **4 层 8 个模块**：



层	模块	输入	输出
感知	FastLIO2	原始雷达点云 + IMU	odom + lidar（已对齐到世界）
建图	VoxelGridMapper	lidar 点云流	global_map（3D 体素点云）
建图	CostMapper	global_map	global_costmap（2D OccupancyGrid）
规划	WavefrontFrontierExplorer	global_costmap + odom	goal_request（自动选探索目标）
规划	ReplanningAStarPlanner	odom + global_costmap + goal_request	nav_cmd_vel
控制	MovementManager	nav_cmd_vel + tele_cmd_vel + clicked_point	cmd_vel + stop_movement
控制	Robot Adapter	cmd_vel	电机指令

新增：Nav Stack 0.1 — 另一条平行路线

上游 commit [2a430b55b](#)（2026-05-09）合并了完整的第二套导航栈 [dimos/navigation/nav_stack/](#)，主要给 **Unitree G1 humanoid** 用，但任何带 LiDAR 的平台都能接。

关键认知：Go2 没有切到这套。 Go2 的 [unitree_go2](#) blueprint 里仍然是老栈。本教程主体（第三~六章）讲的是老栈，依然完全适用。本节只做新老对比，让你明白"看到了 Nav Stack 但跟我看的教程对不上"是怎么回事。

为什么要做新栈？

老栈的几个痛点：

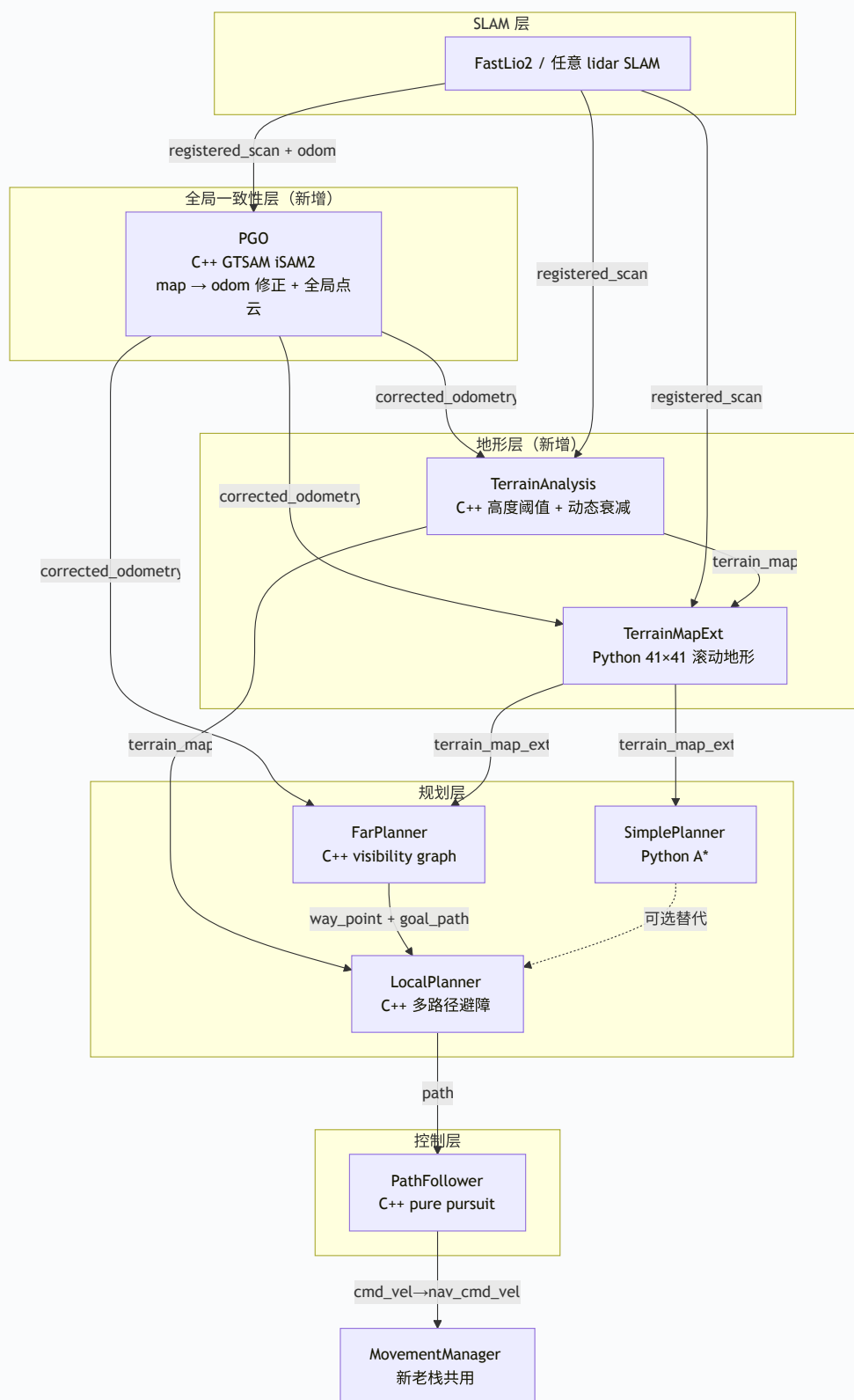
- 1. **Python A* 单线程算路径**：100m × 100m 的大地图扛不住
- 2. **没有回环检测**：长时间跑，里程计会漂移
- 3. **OccupancyGrid 抹掉 Z 轴信息**：humanoid 上下楼梯、跨过低矮障碍时不够用
- 4. **VoxelGridMapper 只累积，不衰减**：动态物体（行人、家具被搬走）会留痕迹

Nav Stack 0.1 的核心思路是：把重计算搬到 C++ NativeModule，把"地形"作为头等公民。

模块对照表

职责	老栈 (Go2 在用)	Nav Stack 0.1 (G1 在用)
里程计修正	无 (直接用 SLAM 给的 odom)	<code>PGO</code> (C++ GTSAM iSAM2 + ICP 回环检测) → <code>corrected_odometry</code>
地形/可走表面	<code>VoxelGridMapper</code> (Open3D VoxelBlockGrid 全局累积)	<code>TerrainAnalysis</code> (C++, 按高度阈值 + 动态衰减输出 <code>terrain_map</code> 点云)
局部地形扩展	无	<code>TerrainMapExt</code> (Python, 41×41 滚动地形格 + BFS 连通性)
障碍/代价表示	<code>CostMapper</code> → 2D <code>OccupancyGrid</code>	直接用 <code>terrain_map</code> 点云 (保留 Z)
全局规划	<code>ReplanningAStarPlanner</code> (Python A* + C++ 加速)	二选一: <code>FarPlanner</code> (C++ visibility graph, 大地图) / <code>SimplePlanner</code> (Python 简化 A*)
局部避障	<code>LocalPlanner</code> (Python 状态机内置)	<code>LocalPlanner</code> (独立 C++ NativeModule, 多路径多扫描融合)
路径跟踪	<code>PController</code> (Python)	<code>PathFollower</code> (C++ pure pursuit + PID)
自主探索	<code>WavefrontFrontierExplorer</code>	<code>TarePlanner</code> (可选)
录制	<code>Recorder</code> (memory2)	<code>NavRecord</code> (专用, 记 14 个导航 topic)
坐标系	只用 <code>map</code>	严格分层: <code>map</code> → <code>odom</code> → <code>body</code> → <code>sensor</code>
输出	<code>nav_cmd_vel</code>	<code>nav_cmd_vel</code> (PathFollower 的 <code>cmd_vel</code> 在 blueprint 里 remap 成这个)
构图方式	手动 <code>autoconnect(VoxelGridMapper, CostMapper, ...)</code>	<code>create_nav_stack(planner="far", use_terrain_map_ext=True, ...)</code> 一行

Nav Stack 0.1 的拓扑



谁在用？

- **G1 onboard / sim:**

```
dimos/robot/unitree/g1/blueprints/navigation/unitree_g1_nav_onboard.py 调用  
create_nav_stack(...)
```

- **Go2: 没动，仍是老栈**

如果你只关心 Go2，**第三~六章完全有效**，直接读下去。如果你要给 G1 改导航或者想了解 Nav Stack 0.1 内部细节，跳到 [7.5 Nav Stack 0.1 完全指南](#)。

三、建图（Mapping） — 让机器人"看见"环境

3.1 第一站：SLAM 提供 odom

dimos 不实现 SLAM，但用一个 **FastLIO2** 模块（`dimos/hardware/sensors/lidar/fastlio2/`）把 SLAM 包成普通 dimos 模块。

FastLIO2（Fast LiDAR-Inertial Odometry 2）是港大开源的 SOTA 雷达 SLAM。Livox MID-360 + IMU → 实时输出 `odom` + `lidar` 帧（每帧已对齐到世界坐标系）。

实例（`mid360_fastlio_voxels` blueprint）：

```
mid360_fastlio_voxels = autoconnect(  
    FastLio2.blueprint(),                # SLAM 进程  
    VoxelGridMapper.blueprint(voxel_size=0.05),  # 体素累积  
    vis_module("rerun", rerun_config={...}),    # 可视化  
)
```

只要有 SLAM 在跑，下游所有模块都能从 `odom` 拿到当前姿态、从 `lidar` 拿到点云。

3.2 第二站：VoxelGridMapper — 把点云累积成 3D 地图

文件：`dimos/mapping/voxels.py`

问题：每帧雷达只能看到附近一块，怎么把多帧拼成一张完整的 3D 地图？

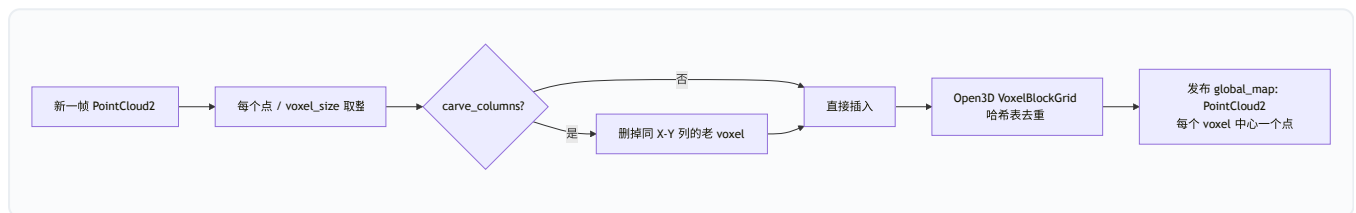
答案：用 Open3D 的 `VoxelBlockGrid`（GPU 加速的 3D 哈希表）。每帧来了：

1. 把每个点除以 voxel size 取整 → 得到 voxel 坐标 `(x, y, z)`
2. 把这个坐标作为 key 插到哈希表里

哈希表自带去重，扫过千百次同一个地方也只占一个 voxel。

Column carving（列裁剪）

如果一个 (x, y) 列的 voxel 在新一帧里出现了，**先把这个列上所有老 voxel 删掉**再插入新 voxel。这样能处理"东西被搬走了"的情况（动态物体不会在地图上留痕迹）。



框架视角看 VoxelGridMapper

`VoxelGridMapper` 继承自上一节升级里的 `StreamModule`：声明一个 `In[PointCloud2]` 和一个 `Out[PointCloud2]`，剩下的输入累积、输出节流由基类自动黏合。

```
class VoxelGridMapper(StreamModule[PointCloud2, PointCloud2]):
    config: VoxelGridMapperConfig
    lidar: In[PointCloud2]          # 雷达原始扫描
    global_map: Out[PointCloud2]    # 累积后的全局体素地图

    def pipeline(self, stream):
        return stream.transform(VoxelMapTransformer(**cfg))
```

2026-05 更新：commit `1737d037e` 修了一个**CUDA 内存泄漏**。Open3D 的 `HashMap` 操作（`carving`、`key_tensor()[idx]`、`find()`、`activate()`）每次都会从设备分配缓冲区，但 Open3D 的 caching allocator **不会主动归还**——长跑下 VRAM 每次调用涨 ~0.8 MB 直到 OOM。现在每次 insert 后会显式释放：

```
python if str(self._dev).startswith("CUDA"): o3c.cuda.release_cache()
```

影响：用 GPU 跑 `VoxelGridMapper` 几小时不再爆显存。CPU 模式不受影响。

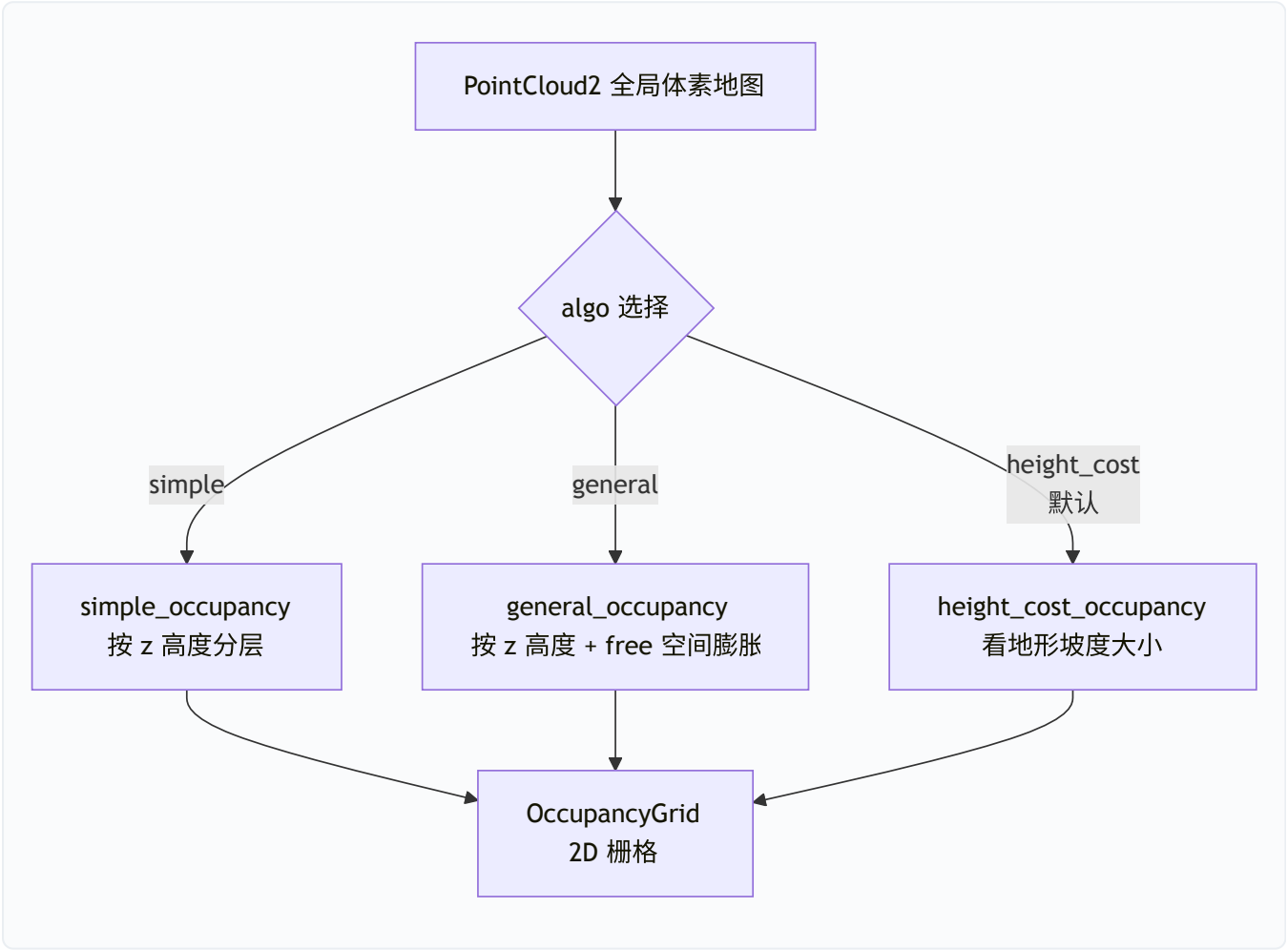
3.3 第三站：CostMapper — 把 3D voxel 拍扁成 2D 占据栅格

文件：`dimos/mapping/costmapper.py` + `dimos/mapping/pointclouds/occupancy.py`

问题：地面机器人不需要 3D，需要"地面上每个格子能不能踩"的 2D 视角。

答案：把 3D voxel 投影到 X-Y 平面，用某种算法判断"这个格子可走/不可走"。

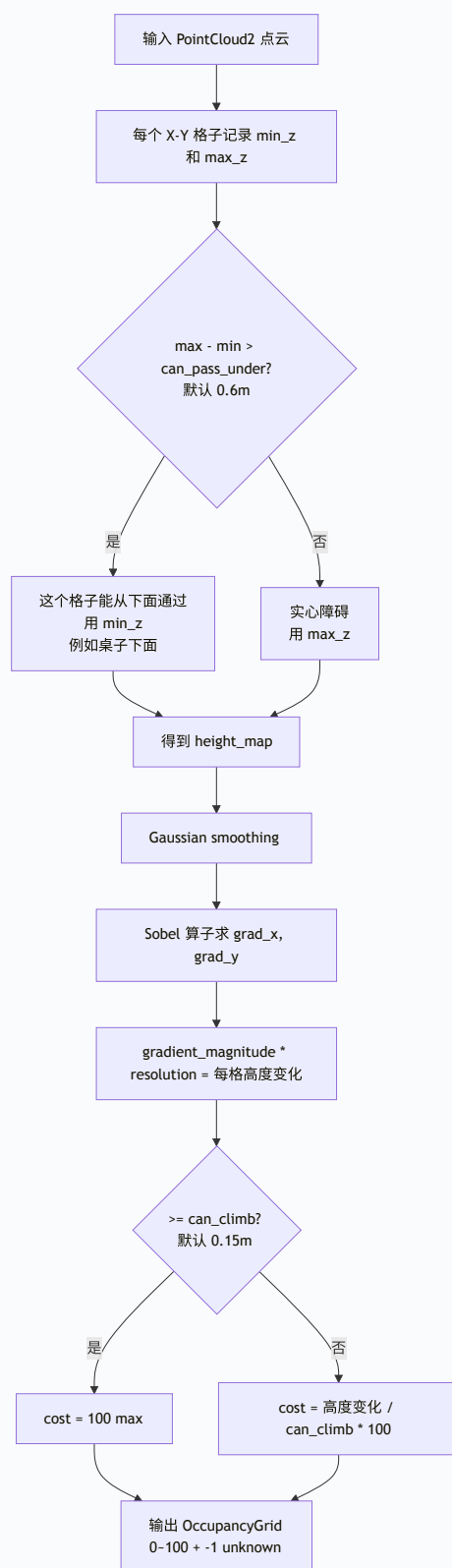
dimos 提供 3 种 occupancy 算法（注册在 `OCCUPANCY_ALGOS`）：



三种算法对比

算法	思路	适合	缺点
<code>simple</code>	高度在 <code>[min_h, max_h]</code> 之间 = 障碍；低于 = free	室内、地面平坦	不分坡度
<code>general</code>	同上 + 把扫到的地面附近一圈也标 free	同上，但 free 空间更确定	算得慢
<code>height_cost</code> 默认	计算每个格子的地形坡度（用 Sobel 算子求高度梯度），坡度越大 cost 越高	户外、有台阶/斜坡的环境	需要好几个超参数

height_cost_occupancy 详解（默认算法）

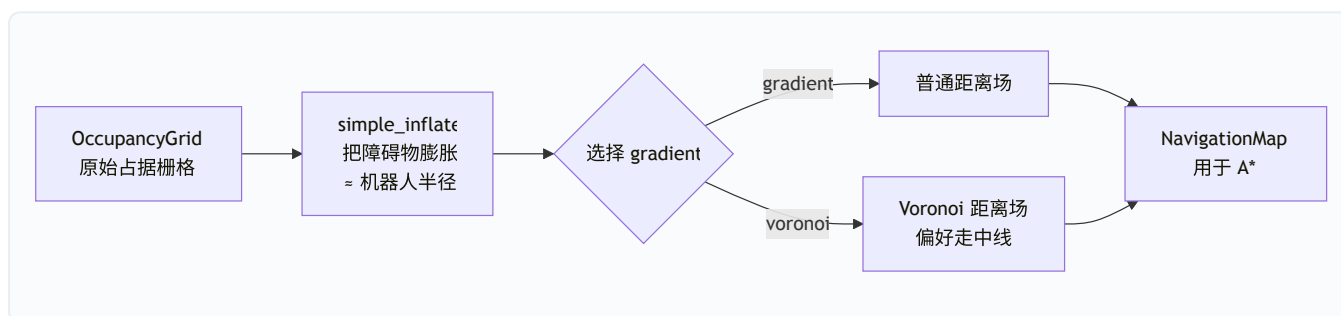


配置项含义：

字段	默认	含义
<code>resolution</code>	0.05	每格 5cm
<code>can_pass_under</code>	0.6	高度差超过这个就算"底下能过"（桌椅下方）
<code>can_climb</code>	0.15	一格内高度变化超过这个就 cost=100（爬不上去）
<code>ignore_noise</code>	0.05	小于这个的高度变化忽略（雷达噪声）
<code>smoothing</code>	1.0	Gaussian 平滑系数

3.4 第四站：把 OccupancyGrid 进一步加工成"导航地图"

`OccupancyGrid` 还不能直接给规划器用，需要两步加工：



障碍物膨胀（`simple_inflate`）

A 把机器人当成一个"点"。但机器人有半径，紧贴墙走会刮到。解决办法：把所有障碍物按机器人半径膨胀*。这样规划出来的"点的路径"自动满足"机器人不撞墙"。

具体实现（`dimos/mapping/occupancy/inflation.py`）：

```

# 把半径转成格子数
cell_radius = ceil(robot_radius / resolution)
# 圆形 kernel
kernel = (x*x + y*y <= cell_radius*cell_radius)
# 用 scipy 二值膨胀
inflated = scipy.ndimage.binary_dilation(occupied, kernel)

```

梯度成本图（`gradient` / `voronoi_gradient`）

膨胀完之后，墙外的格子还是要么 0 要么 100。希望"离墙越远越便宜"，所以叠一个距离场：

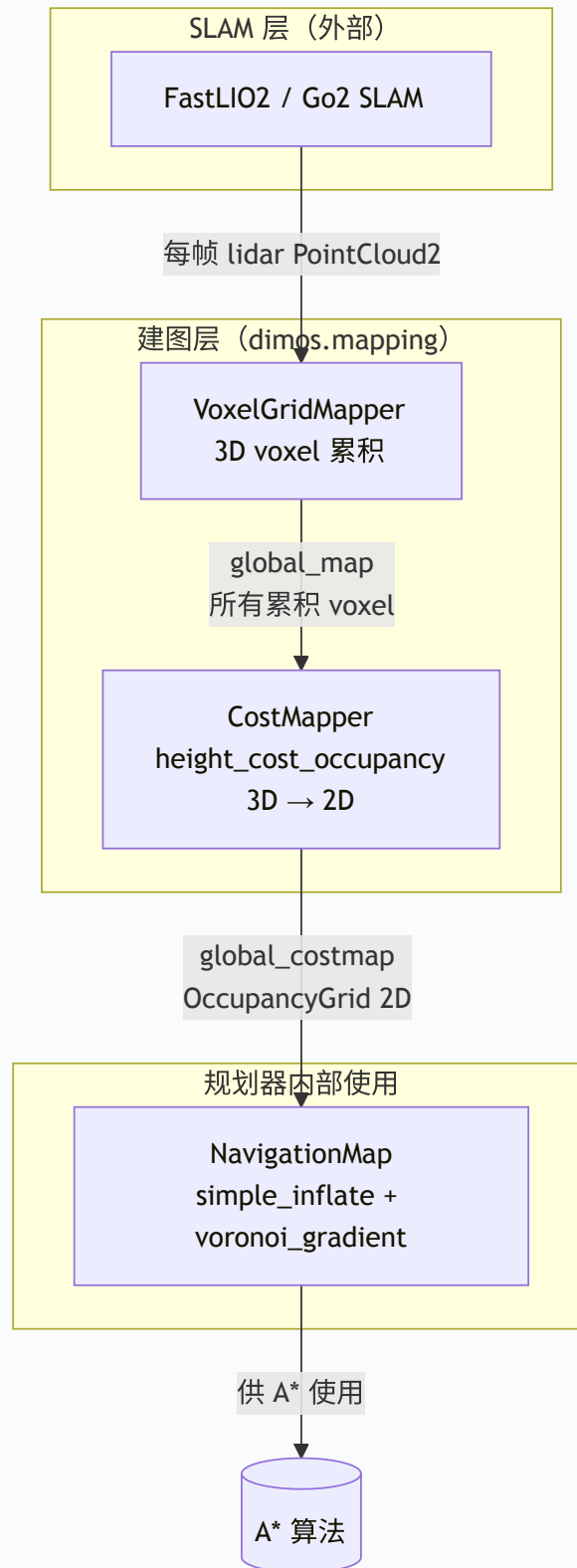
- `gradient`：每个 free 格子的 cost = $1 - \text{clip}(\text{到最近障碍距离} / \text{max_distance}, 0, 1) \times 100$
- `voronoi_gradient`：偏好走两侧障碍中间的"中线"（Voronoi 图的边）

最终得到的就是"在 cost 上跑 A*"用的图。

dimos 在 `GlobalPlanner` 里同时维护两张 `NavigationMap`:

- `_navigation_map` (Voronoi): 远距离用, 偏好走中线
- `_navigation_map_near` (gradient): 近距离用 ($< 1.5\text{m}$), 偏好直来直去

3.5 完整的"建图链"流程



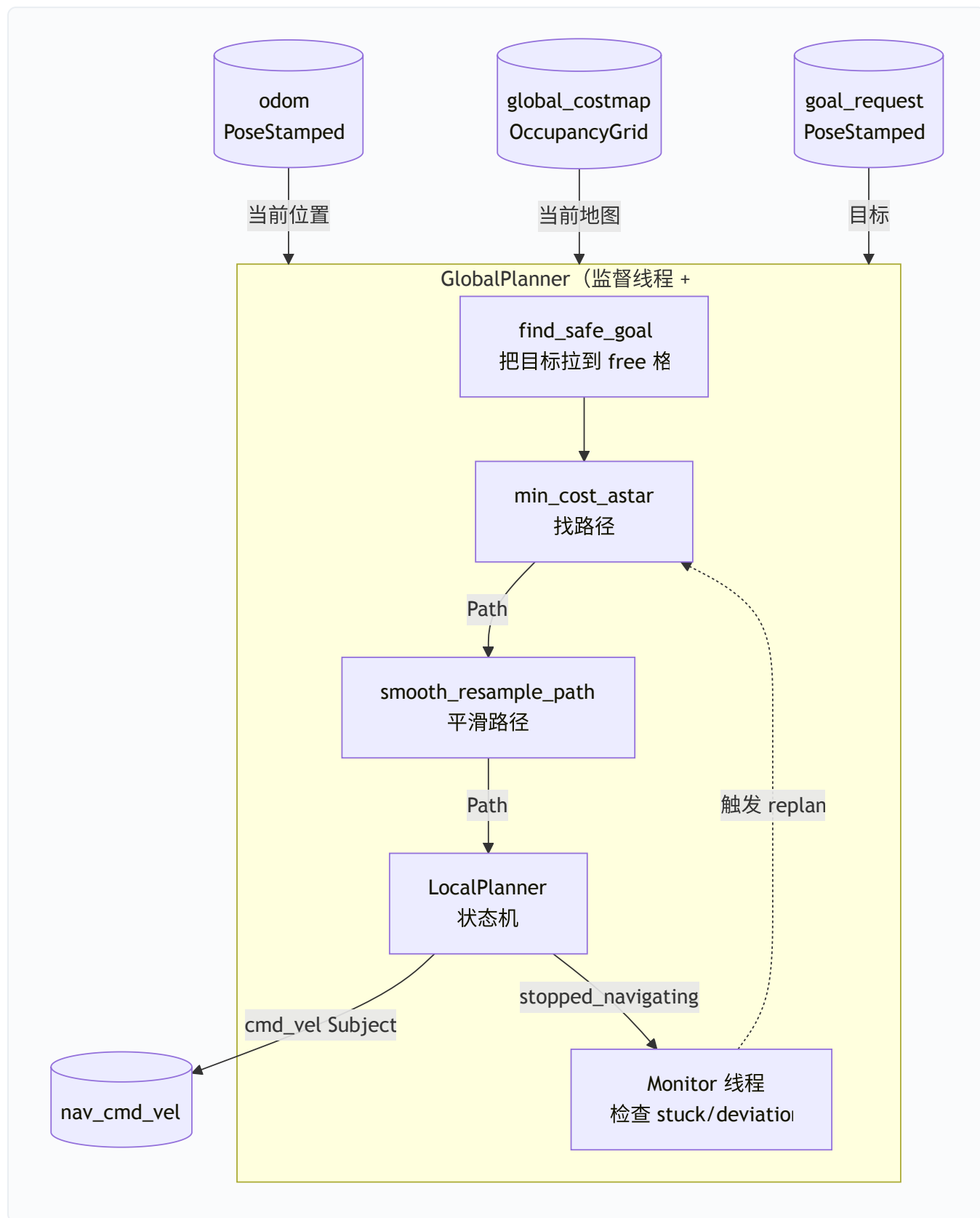
四、导航（Navigation） — 让机器人"会走路"

dimos 的导航主力是 `ReplanningAStarPlanner`（`dimos/navigation/replanning_a_star/`）。它不是单一类，而是 6 个文件 6 个职责的小系统：

文件	类	职责
<code>module.py</code>	<code>ReplanningAStarPlanner</code>	dimos Module 包装，端口和生命周期
<code>global_planner.py</code>	<code>GlobalPlanner</code>	协调器：监听 odom/goal/costmap，调度全局规划 + 局部跟随
<code>min_cost_astar.py</code>	<code>min_cost_astar()</code>	A* 算法本体（含 C++ 加速实现）
<code>local_planner.py</code>	<code>LocalPlanner</code>	状态机 + 沿路径跟踪 + 控制器调用
<code>navigation_map.py</code>	<code>NavigationMap</code>	把 OccupancyGrid 加工成 cost map
<code>controllers.py</code>	<code>PController</code> / <code>PdController</code>	把"位置误差"转 Twist
<code>goal_validator.py</code>	<code>find_safe_goal()</code>	把不可达目标拉到附近可达点
<code>position_tracker.py</code>	<code>PositionTracker</code>	检测"卡住"
<code>path_clearance.py</code>	<code>PathClearance</code>	检测前方动态障碍
<code>replan_limiter.py</code>	<code>ReplanLimiter</code>	防止无限重规划

下面分四个层次讲清楚。

4.1 顶层数据流



4.2 第一步：A* 在做什么

文件： `dimos/navigation/replanning_a_star/min_cost_astar.py`

接口： `min_cost_astar(costmap, goal, start) -> Path | None`

核心算法（伪代码版本）：

```
open_set = priority_queue { (f=0, start) }
closed_set = {}
came_from = {}
g_score = { start: 0 }

while open_set 非空:
    current = open_set.pop_min()

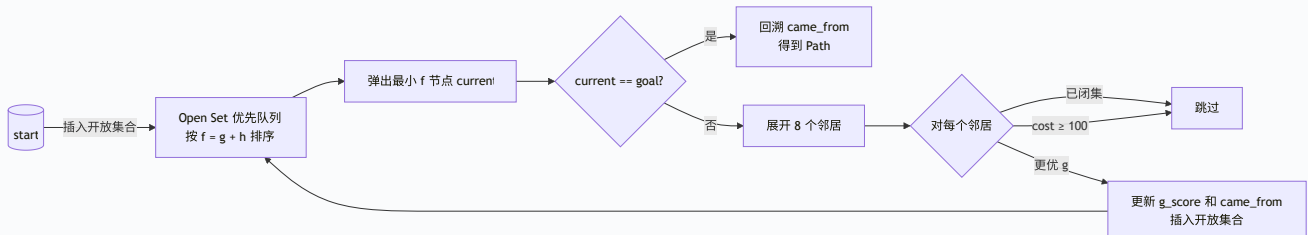
    if current == goal:
        return reconstruct_path(came_from, current)

    closed_set.add(current)

    for neighbor in 8 个邻居:
        if neighbor 已 closed 或 cost >= 100: continue
        cell_cost = neighbor 在 costmap 上的值 (0~99, UNKNOWN 给惩罚)
        tentative_g = g_score[current] + cell_cost # ← 关键: 累积 cost
        if tentative_g < g_score.get(neighbor, ∞):
            came_from[neighbor] = current
            g_score[neighbor] = tentative_g
            f = tentative_g # ← cost 优先, 距离次之
            heappush(open_set, (f, neighbor))
```

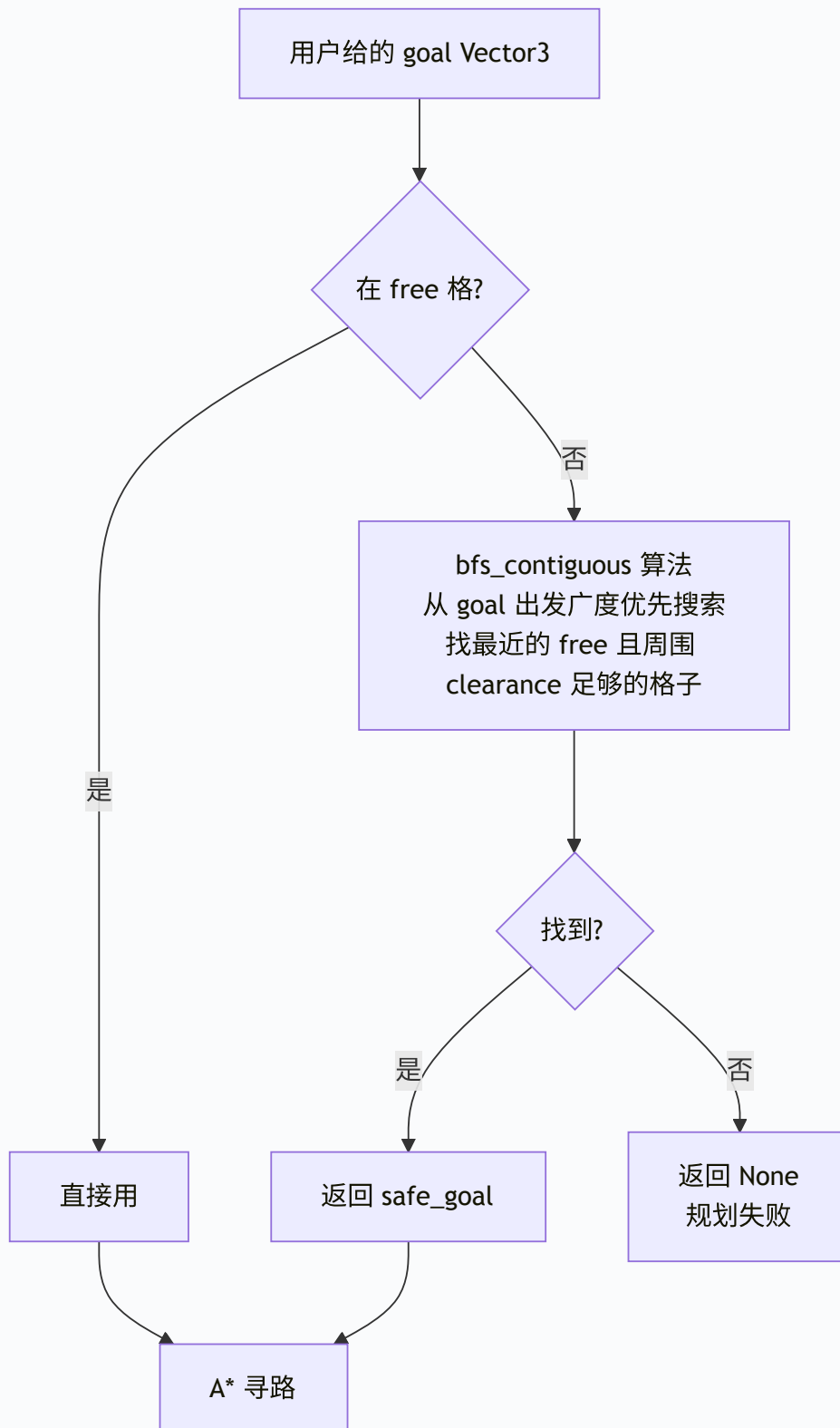
dimos 的两个特别之处：

1. 优先级是 **(累积cost, 累积距离 + 启发距离)**，是个二元组比较，先比 cost 后比距离。这意味着 A* 优先找"安全"的路，再在同等安全度下找"最短"。
2. C++ 加速： `min_cost_astar.py:25` 试图 import `min_cost_astar_cpp`（一个 pybind11 编译的 C++ 实现），如果有就用 C++ 版（快 10x+），fallback 到 Python。
3. 启发函数用 **Octile 距离**（适合 8-邻接，比欧氏距离更准）。



4.3 第二步：safe goal — 智能纠偏目标

如果你点的目标恰好在墙里、桌子上，A* 直接 fail。 `find_safe_goal` 解决这个：



`min_clearance = robot_rotation_diameter / 2`：要求新目标周围至少有半个机器人转身半径的余量，免得到了之后转不开。

4.4 第三步：path smoothing — 让路径不那么折角

A* 输出的路径是格子坐标序列，常常 zigzag。直接给控制器，机器人会忽左忽右"画蛇"。

`smooth_resample_path` (`dimos/mapping/occupancy/path_resampling.py`)：

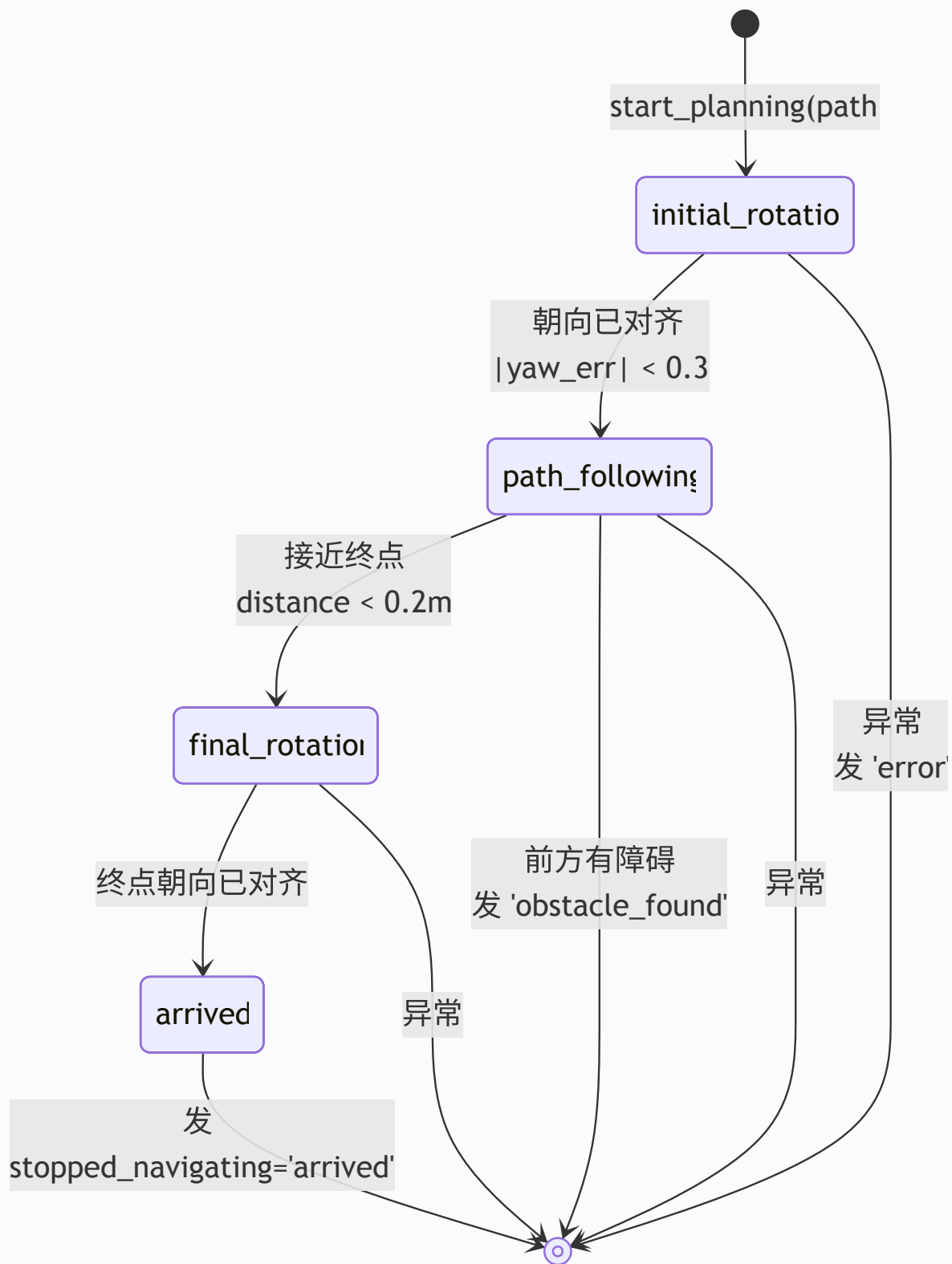
1. 用线性插值上采样到 100+ 个点
2. 用 `uniform_filter1d` 滑窗平均（默认窗口 100）平滑 X、Y 坐标
3. 重新按指定 spacing（默认 0.1m）重采样
4. 起点终点保持不动
5. 自动给每个点算朝向（朝向下一点）

效果：



4.5 第四步：LocalPlanner 状态机 — 沿路径走

`LocalPlanner._loop()` (`local_planner.py:163`) 是个 4 状态有限状态机，10Hz 控制循环：



每个状态在做什么：

状态	用什么算 cmd_vel
<code>initial_rotation</code>	<code>controller.rotate(yaw_error)</code> — 原地转到第一个 waypoint 的朝向
<code>path_following</code>	找路径上当前最近点 → 算 lookahead 点 → <code>controller.advance(lookahead, odom)</code>
<code>final_rotation</code>	<code>controller.rotate(yaw_error)</code> — 转到目标的最终朝向
<code>arrived</code>	发 <code>Twist()</code> (零速) + 通知 GlobalPlanner

4.6 第五步：PController — 从 lookahead 点算 Twist

文件: `dimos/navigation/replanning_a_star/controllers.py`

```
def advance(self, lookahead_point, current_odom) -> Twist:
    direction = lookahead_point - current_pos
    desired_yaw = arctan2(dy, dx)
    yaw_error = desired_yaw - robot_yaw

    angular_v = k_angular * yaw_error          # P 控制

    if |yaw_error| > 90°:
        return Twist(linear=0, angular=angular_v) # 大角度先原地转

    # 朝向差越大走越慢
    linear_v = max_speed * (1 - |yaw_error| / 90°)
    return Twist(linear=linear_v, angular=angular_v)
```

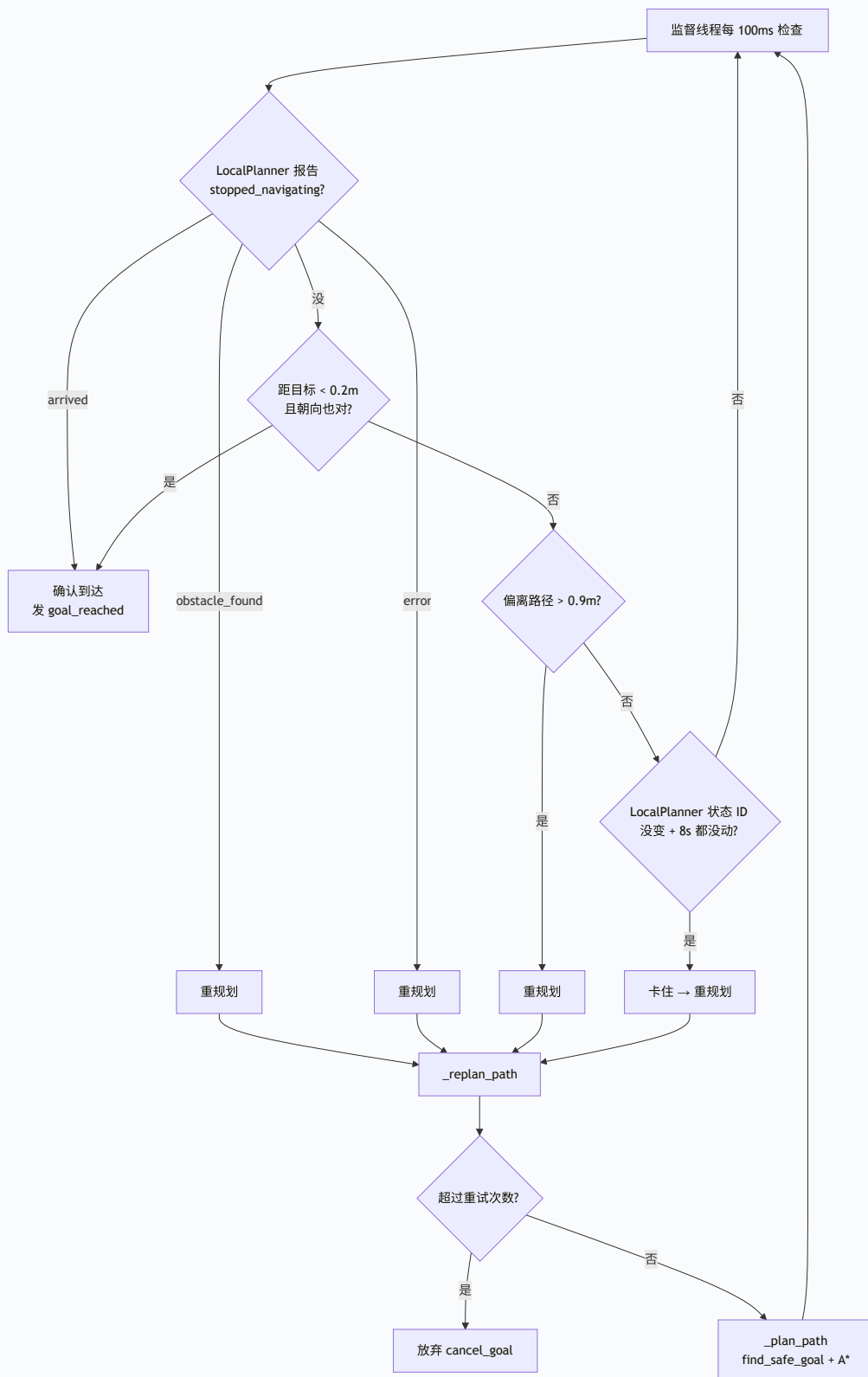
直观规则:

- 朝向差 > 90° → 只转不走 (rotate-then-drive)
- 朝向差小 → 边走边纠
- 离 lookahead 点越歪, 线速度越慢

还有个 `PdController` (PD 控制, 加了导数项), 用于减小震荡, 但默认用的是 P。

4.7 第六步: 重规划 — 路上发生意外了怎么办

`GlobalPlanner._thread_entrypoint()` 是个监督线程, 10Hz 跑下面这套逻辑:



4 个触发重规划的条件：

1. LocalPlanner 报告 **obstacle_found**：PathClearance 在前方扫到新障碍
2. 偏离路径 > 0.9m：被外力推开 / 控制器漂了
3. 卡住：PositionTracker 在 8 秒窗口内位置变化 < 0.4m

4. 接收到新 goal: 直接 `_plan_path()`

`ReplanLimiter` 防止无限重规划: 每 N 次重规划必须实际向目标推进 X 米, 否则就放弃。

4.8 第七步: 和 MovementManager 联动

`MovementManager` 是导航/遥控/点击 三路命令的总线。文件位置

`dimos/navigation/movement_manager/movement_manager.py` (注意: 早期叫

`dimos/navigation/smart_nav/modules/movement_manager/`, commit `2a430b55b` 已经搬到现在的位置; 如果你看老文档/老 import 报错, 就是这个原因)。

端口

```
class MovementManager(Module):
    # 输入
    clicked_point: In[PointStamped] # 用户在 viewer 上点的位置
    nav_cmd_vel: In[Twist] # 来自 ReplanningAStarPlanner
    tele_cmd_vel: In[Twist] # 来自 KeyboardTeleop / 摇杆

    # 输出
    goal: Out[PointStamped] # 转发点击作为目标
    way_point: Out[PointStamped] # 当前 waypoint (也用于取消信号)
    cmd_vel: Out[Twist] # 实际下发给机器人的速度
    stop_movement: Out[Bool] # 通知 planner 停止
```

三个关键行为

1. 点击转目标 + 安全检查

```
MAX_CLICK_HORIZONTAL_M = 500.0 # 防止点到无穷远
MAX_CLICK_VERTICAL_M = 50.0
```

收到 `clicked_point` 时先校验 NaN/Inf 和量级, 通过了再同时发 `way_point` + `goal`。

2. 遥控优先 + 冷却期

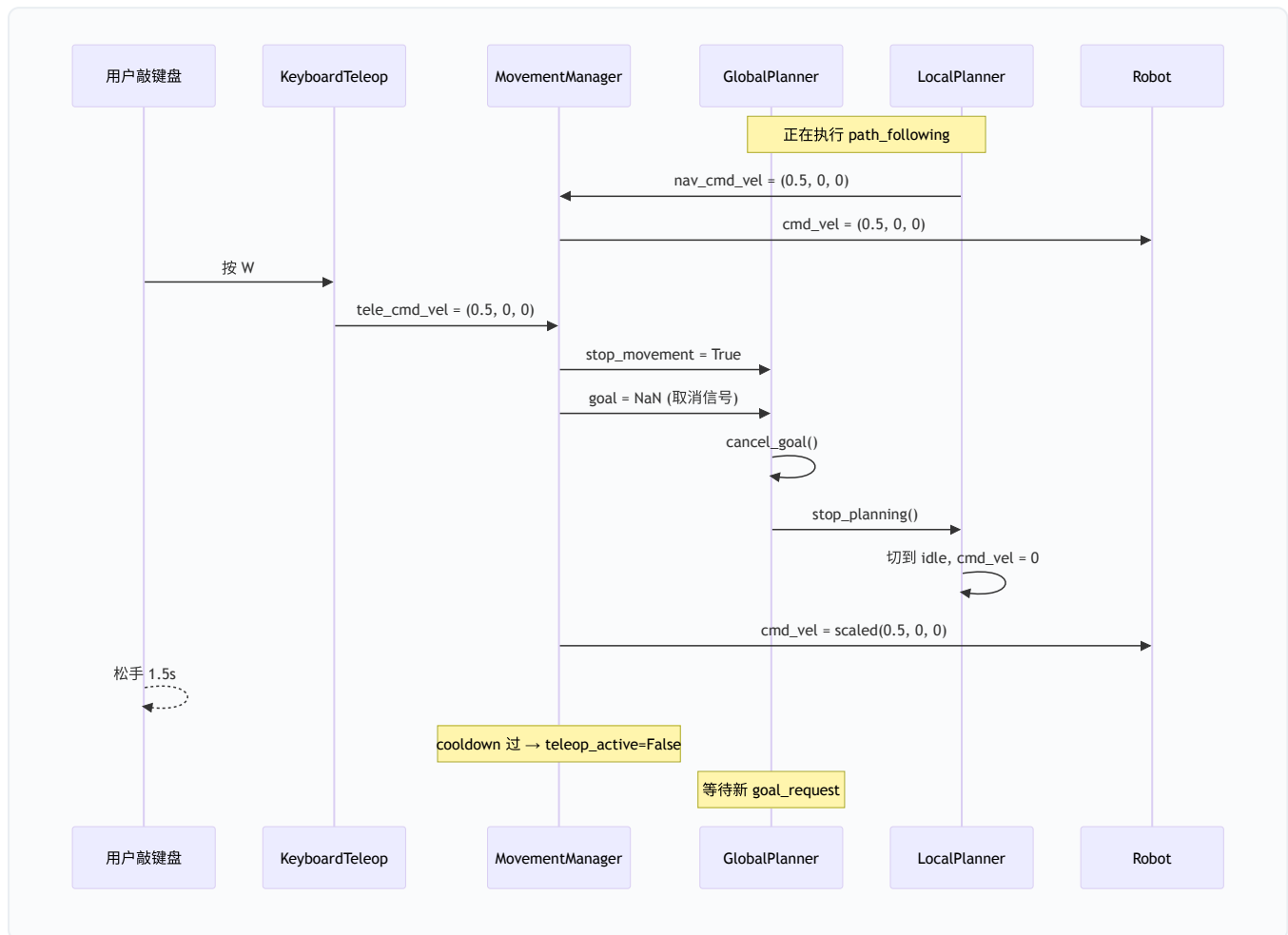
收到 `tele_cmd_vel` 时立刻: - 发 `stop_movement=True` 让 planner 取消 - 发一个坐标全 NaN 的 `PointStamped` 作为 goal ("安全回退"语义, 让任何还在跑的 planner 知道"放弃当前目标") - 把遥控速度按 `tele_cmd_vel_scaling` 缩放后发到 `cmd_vel`

之后 1 秒 (`tele_cooldown_sec`) 内进来的 `nav_cmd_vel` 全部忽略, 避免 planner 抢方向盘。

3. 自动 nav 模式

cooldown 过了 + 没有遥控输入 → `nav_cmd_vel` 直接转发到 `cmd_vel`。

完整时序



额外说明：新栈 Nav Stack 0.1 的 `PathFollower.cmd_vel` 在 blueprint 里被 remap 成 `nav_cmd_vel`，所以 **MovementManager** 同时服务老栈和新栈——这就是为什么它没放在 `replanning_a_star/` 目录下，而是单独一个模块。

五、自主探索（Frontier Exploration）— 让机器人"自己出门"

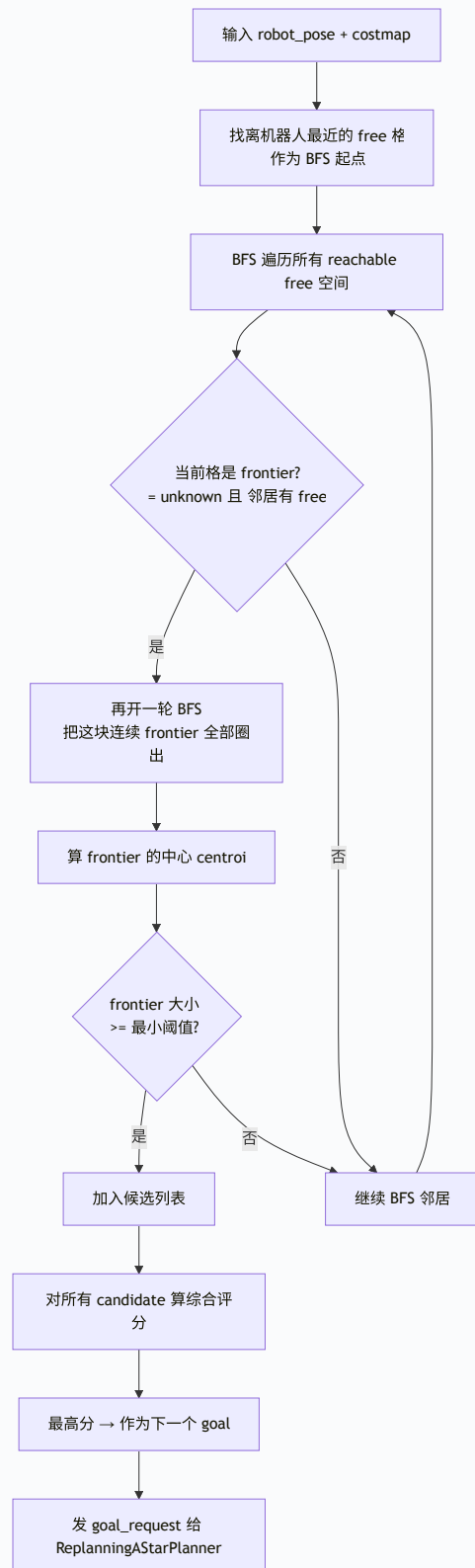
文件：`dimos/navigation/frontier_exploration/wavefront_frontier_goal_selector.py`

5.1 Frontier 是什么

Frontier = 已知 free 空间和未知空间的**边界**。机器人想"探索"，就要往这些边界走 — 走过去之后那块未知就变成已知了。



5.2 Wavefront 算法步骤



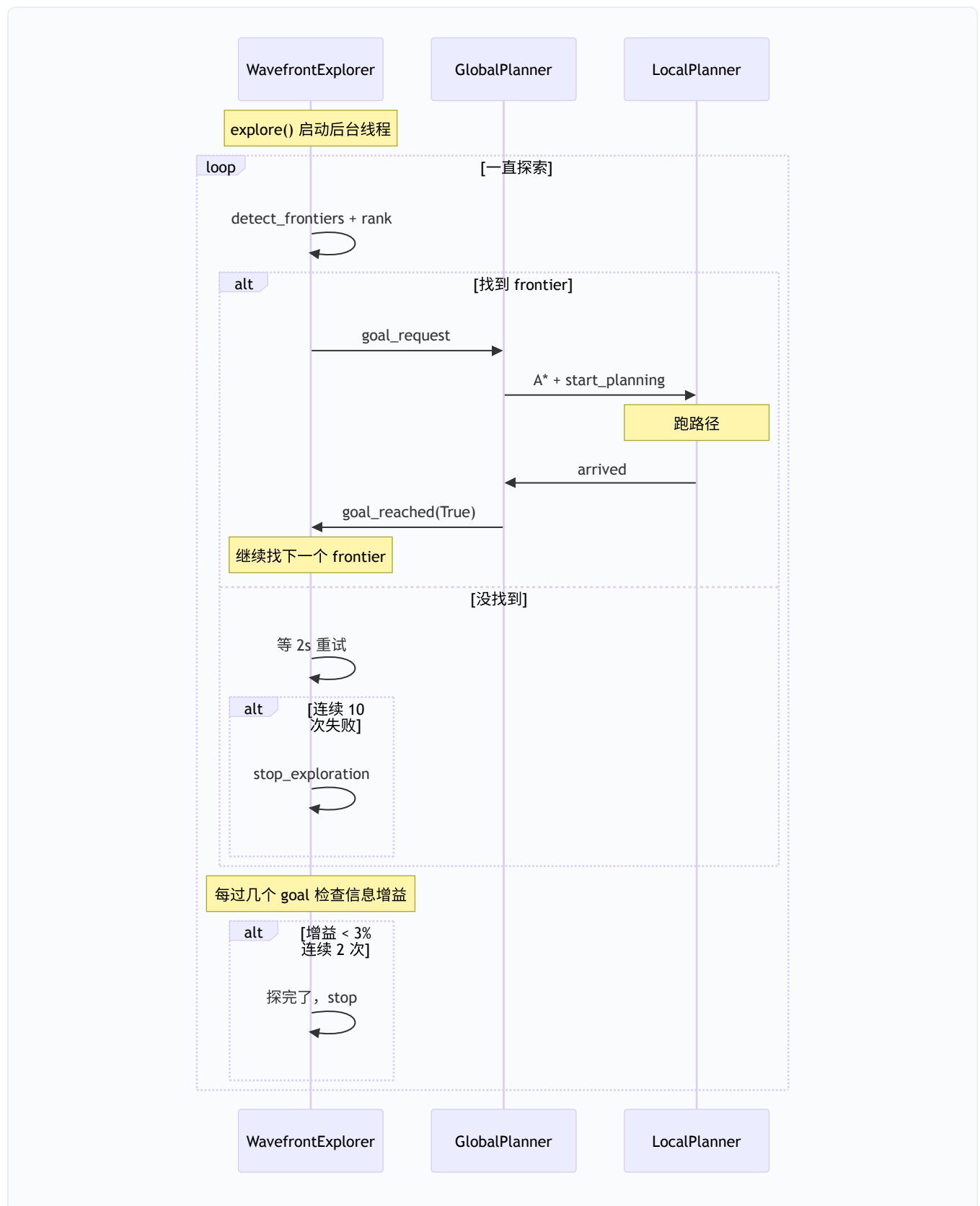
5.3 综合评分（5 个权重）

`_compute_comprehensive_frontier_score` :

维度	权重	含义
info_gain	30%	frontier 越大，能新看到的区域越多
explored_distance	30%	离已经探索过的目标越远越好（避免来回打转）
distance	20%	适中距离最好（5m 默认 lookahead）
obstacles	15%	离障碍物越远越安全
momentum	5%	偏好"上次方向"，避免乱转

最高分胜出，发给规划器。

5.4 与 Planner 的协作



agent 可以通过 skill 启停:

```
@skill
def begin_exploration(self) -> str: ...
@skill
def end_exploration(self) -> str: ...
```

5.5 PatrollingModule — 已知地图上的"巡逻"

文件: `dimos/navigation/patrolling/module.py`

问题: 地图已经建好了 (或者用户希望机器人在已知区域里反复巡查), 怎么让它自动循环挑下一个目标点?

答案: `PatrollingModule` 内置三种 `PatrolRouter` :

Router	行为	适合
<code>RandomPatrolRouter</code>	在 free 区域随机采样	监控类应用
<code>CoveragePatrolRouter</code> 默认	优先走"很久没访问"的格子 (看 <code>VisitationHistory</code>)	全覆盖巡逻
<code>FrontierPatrolRouter</code>	同时考虑 frontier, 类似第五章	边巡边探

```
@skill
async def start_patrol(self) -> str: ...
@skill
async def stop_patrol(self) -> str: ...
```

2026-05 更新 (commit `3bda85b9f`) : `PatrollingModule` 从 `threading + Event` 改造成 `asyncio` :

- 巡逻循环 `_patrol_loop` 现在是 `async def` , `await self._goal_reached_event.wait()` 替代旧的 `threading.Event.wait()`
- skill 方法标 `async def` , agent 调用 `start_patrol` / `stop_patrol` 时不再阻塞 event loop
- 三个回调 (`handle_odom` , `handle_global_costmap` , `handle_goal_reached`) 改为 `async def` 自动挂到 reactive stream
- 停止巡逻时 `_stop_patrolling` 会 `await self._patrol_task` 优雅取消

行为不变, 只是不再吃线程池配额、不再阻塞。这是 `dimos Module` 系统支持 `async` 的必然演进。

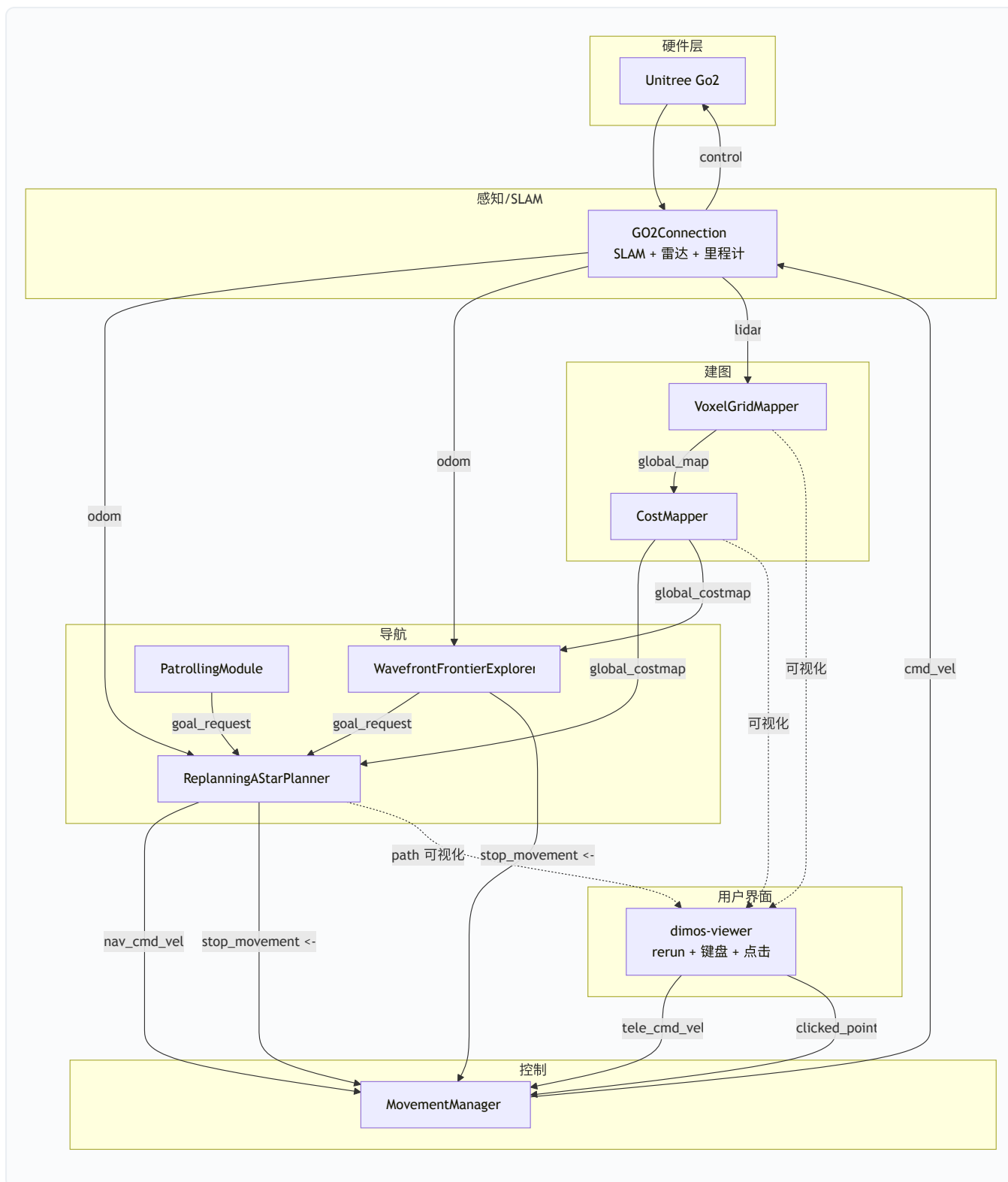
六、端到端实战 — 用 Go2 做一次完整任务

6.1 启动命令

```
# 真机
dimos run unitree-go2 --robot-ip 192.168.123.161

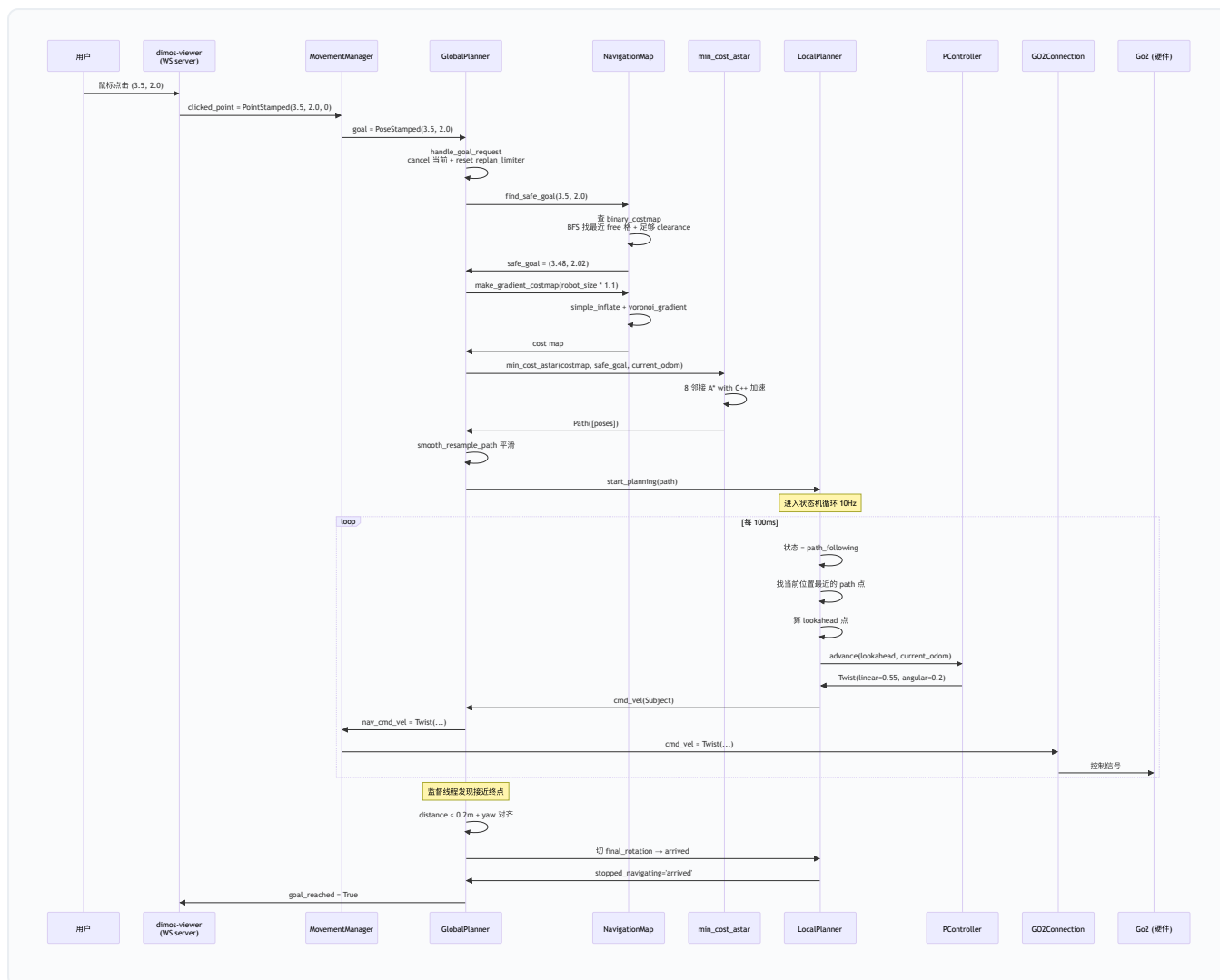
# 仿真
dimos --simulation run unitree-go2-agentic-sim
```

`unitree-go2` blueprint (`dimos/robot/unitree/go2/blueprints/smart/unitree_go2.py`) 的拓扑:



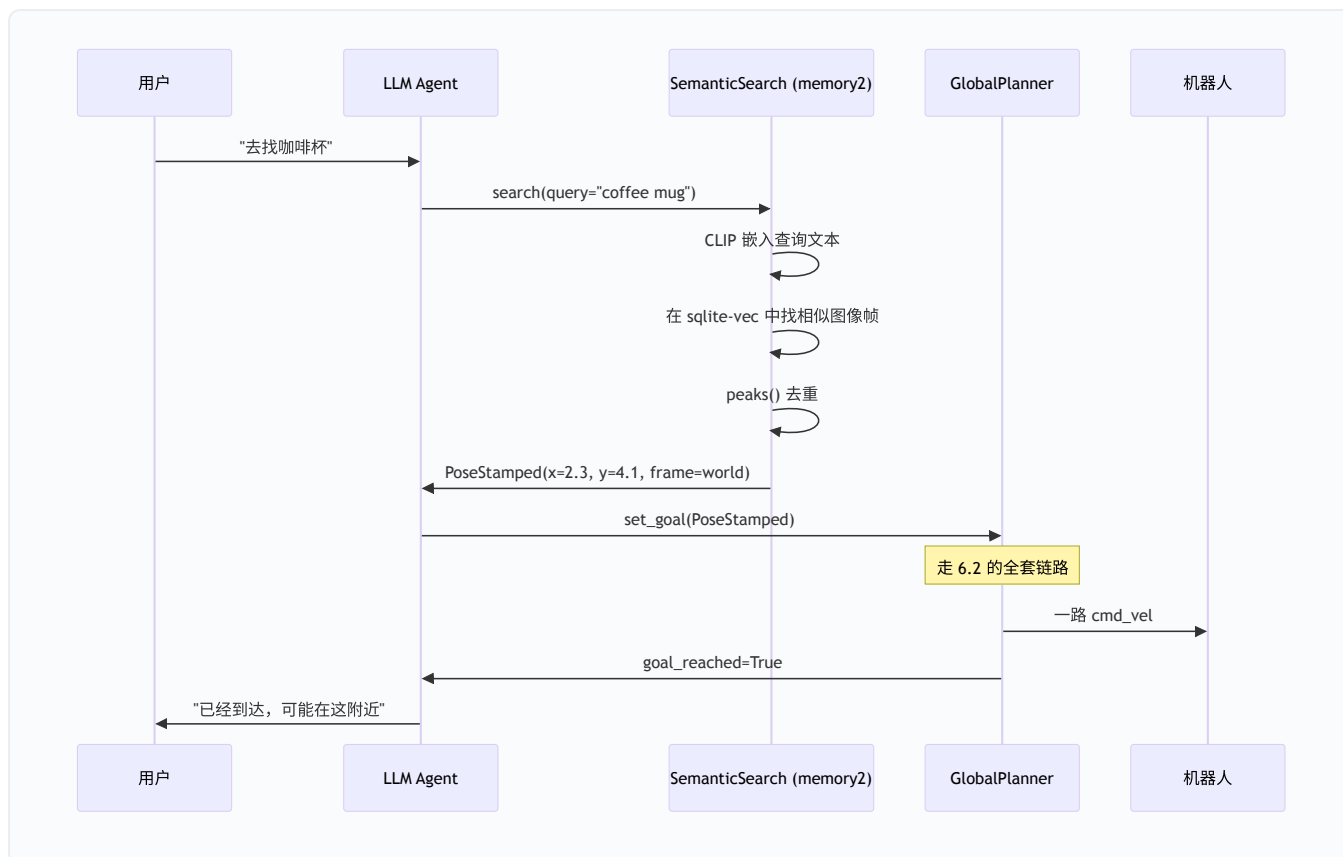
6.2 一次"点哪走哪"的完整数据流

假设你启动了系统，在 rerun 上点击地图 (3.5m, 2.0m) 一个位置：



6.3 Agent 用语义检索找东西的完整链路

如果你用了 `unitree-go2-agentic` 这种带 agent 的 blueprint，agent 收到"去找咖啡杯"指令的全程：



七、扩展点和延伸阅读

7.1 自己写一个 occupancy 算法

只需在 `dimos/mapping/pointclouds/occupancy.py` 里：

```

@dataclass(frozen=True)
class MyConfig(OccupancyConfig):
    my_param: float = 1.0

def my_occupancy(cloud: PointCloud2, **kwargs) -> OccupancyGrid:
    cfg = MyConfig(**kwargs)
    points, _ = cloud.as_numpy()
    # ... 你的算法 ...
    return OccupancyGrid(grid=..., resolution=cfg.resolution, origin=..., frame_id=...)

OCCUPANCY_ALGOS["my_algo"] = my_occupancy
  
```

然后在 blueprint 里：

```
CostMapper.blueprint(config=Config(algo="my_algo", config=MyConfig(my_param=2.0)))
```

7.2 自己写一个 controller

实现 `dimos/navigation/replanning_a_star/controllers.py:Controller` 协议：

```
class MyController:
    def advance(self, lookahead_point, current_odom) -> Twist: ...
    def rotate(self, yaw_error) -> Twist: ...
    def reset_errors(self) -> None: ...
    def reset_yaw_error(self, value) -> None: ...
```

在 `LocalPlanner.__init__` 把 `self._controller` 替换成你的实现。

7.3 替换 SLAM 后端

只要任何模块对外暴露：

- `odom: Out[PoseStamped]`
- `lidar: Out[PointCloud2]` (点云已对齐到 world frame)

就能无缝接入下游建图链。dimos 已支持的：

- FastLIO2 (`dimos/hardware/sensors/lidar/fastlio2/`)
- Unitree Go2 内置 SLAM (`dimos/robot/unitree/go2/connection.py`)
- MuJoCo 仿真 (`dimos/simulation/engines/mujoco_sim_module.py`)

7.4 推荐入门资料

如果想从 dimos 跳出去系统学：

- **SLAM**：港大 FastLIO2 论文 (dimos 用的就是这个的实现)，或 Cartographer (Google) 入门
- **A* 与改进**：D / D Lite / RRT / Hybrid A
- **costmap**：ROS Navigation 的 costmap_2d 包文档 (dimos 的设计深受其影响)
- **控制**：Pure Pursuit (lookahead 控制的鼻祖)、Stanley Controller、TEB Local Planner
- **Pose Graph Optimization**：GTSAM 教程、《Probabilistic Robotics》第 11 章

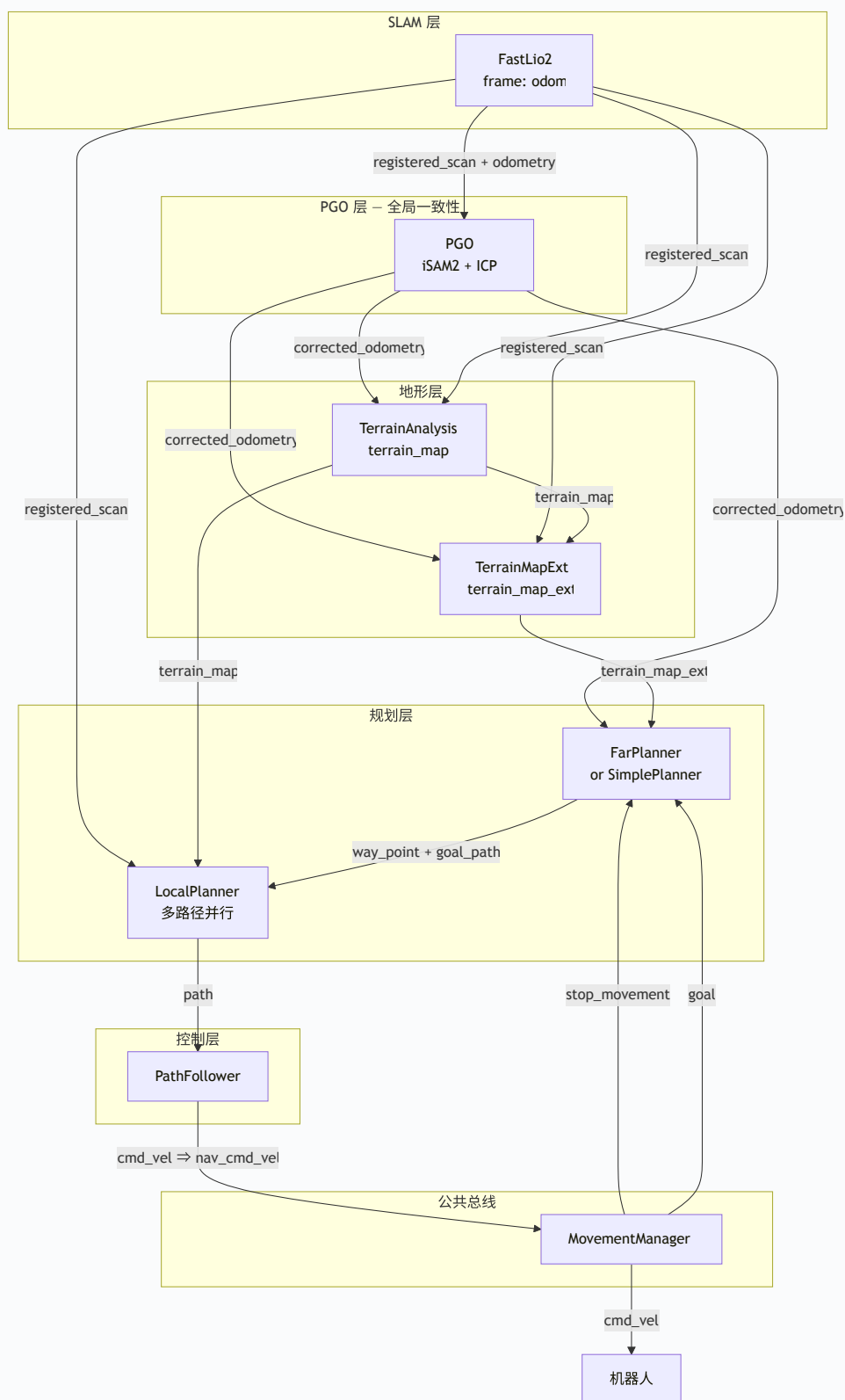
7.5 Nav Stack 0.1 完全指南

这是 commit [2a430b55b](#)（2026-05-09）合并的全新导航栈。**注意：Go2 没用，G1 在用。**本节只在你需要给 G1 改导航 / 在自己平台上接 Nav Stack 时再读。

7.5.1 八个模块速览

模块	父类	实现语言	一句话职责
<code>PGO</code>	<code>NativeModule</code>	C++（GTSAM iSAM2 + PCL ICP）	回环检测，输出 <code>corrected_odometry</code> + <code>map→odom</code> TF
<code>TerrainAnalysis</code>	<code>NativeModule</code>	C++	雷达扫描 → <code>terrain_map</code> 点云（高度阈值 + 动态衰减）
<code>TerrainMapExt</code>	<code>Module</code>	Python	41×41 滚动地形格 + BFS 连通性 → <code>terrain_map_ext</code>
<code>FarPlanner</code>	<code>NativeModule</code>	C++	visibility graph 全局规划， 大地图优势
<code>SimplePlanner</code>	<code>Module</code>	Python	简化 A*，从 <code>terrain_map_ext</code> 直接构 2D costmap
<code>LocalPlanner</code>	<code>NativeModule</code>	C++	多路径并行评估的局部避障，输出 <code>path</code>
<code>PathFollower</code>	<code>NativeModule</code>	C++	pure pursuit + PID yaw 控制，输出 <code>cmd_vel</code>
<code>TarePlanner</code>	<code>NativeModule</code>	C++	TARE 自主探索（可选）
<code>NavRecord</code>	<code>Recorder</code>	Python	把 14 个导航 topic 录到 memory2 SQLite

7.5.2 数据流（完整版）



7.5.3 一句话启动

```
from dimos.navigation.nav_stack.main import create_nav_stack
from dimos.hardware.sensors.lidar.fastlio2.module import FastLio2
from dimos.navigation.movement_manager.movement_manager import MovementManager
from dimos.core.coordination.blueprints import autoconnect

my_nav_stack = autoconnect(
    FastLio2.blueprint(),
    create_nav_stack(
        planner="far",          # "far" 或 "simple"
        use_terrain_map_ext=True,
        vehicle_height=1.5,     # G1 的身高
        max_speed=1.0,
        replan_rate=0.5,        # SimplePlanner 用
        record=False,           # True 启用 NavRecord
    ),
    MovementManager.blueprint(),
)
```

7.5.4 关键概念：四层坐标系

旧栈所有数据都标 `frame_id="map"`。Nav Stack 0.1 严格分层
(`dimos/navigation/nav_stack/frames.py`):

```
map      ← PGO 修正后的全局一致坐标（不漂移，但有跳变）
|  PGO 发布 map→odom TF
↓
odom     ← 里程计本地坐标（连续，但会漂移）
|  机器人本体
↓
body     ← 机器人重心
|  传感器外参
↓
sensor   ← 雷达坐标系
```

PGO 启动时会先发一个 identity TF ($\text{map} \equiv \text{odom}$)，让下游模块能立刻 query $\text{map} \rightarrow \text{body}$ ，不必等第一次回环。

7.5.5 各模块端口表

PGO

方向	端口	类型	说明
In	registered_scan	PointCloud2	已对齐到 odom 的扫描
In	odometry	Odometry	SLAM 给的本地里程计
Out	corrected_odometry	Odometry	应用 map→odom 修正后的里程计
Out	global_map	PointCloud2	全局点云（已应用所有回环修正）
Out	pgo_tf	Odometry	map→odom TF（同时也 publish 到 dimos TF buffer）

TerrainAnalysis (C++ NativeModule, 30+ 个配置项)

方向	端口	类型	说明
In	registered_scan	PointCloud2	雷达扫描
In	odometry	Odometry	用 corrected_odometry remap
Out	terrain_map	PointCloud2	带"地面高度差"的点云

关键配置：

字段	默认	含义
scan_voxel_size	0.05	雷达扫描下采样体素
terrain_voxel_size	0.2	地形格体素
terrain_voxel_half_width	10	滚动窗口半宽（共 21×21 格）
obstacle_height_threshold	0.1	高于这个 = 障碍
vehicle_height	1.5	机器人高度（G1）
decay_time	1.0	旧观测衰减时间（秒）
clear_dynamic_obstacles	True	是否清除动态障碍

FarPlanner (C++, visibility graph)

方向	端口	类型	说明
In	terrain_map_ext	PointCloud2	优先用
In	terrain_map	PointCloud2	备选
In	registered_scan	PointCloud2	用于即时避障判断
In	odometry	Odometry	corrected_odometry
In	goal	PointStamped	来自 MovementManager
In	stop_movement	Bool	取消
Out	way_point	PointStamped	给 LocalPlanner 的下一个 waypoint
Out	goal_path	Path	全局路径（仅可视化）
Out	graph_nodes	GraphNode3D	visibility graph 节点（可视化）
Out	graph_edges	LineSegments3D	graph 边
Out	contour_polygons	ContourPolygons3D	障碍轮廓
Out	nav_boundary	PolygonStamped	导航边界

LocalPlanner (C++, 核心避障)

方向	端口	类型	说明
In	registered_scan	PointCloud2	实时扫描
In	odometry	Odometry	corrected_odometry
In	terrain_map	PointCloud2	地形
In	way_point	PointStamped	来自 FarPlanner/SimplePlanner
In	goal_pose	PoseStamped	终点姿态
In	joy_cmd	Twist	摇杆
In	speed	Float32	限速
In	navigation_boundary	PolygonStamped	不能跨过的边界
In	added_obstacles	PointCloud2	手动加障碍
In	cancel_goal	Bool	取消
Out	path	Path	局部路径 (给 PathFollower)
Out	effective_cmd_vel	Twist	直接控制 (绕过 PathFollower)
Out	free_paths	PointCloud2	候选无碰撞路径 (可视化)
Out	slow_down	Int8	减速等级
Out	goal_reached	Bool	到达

PathFollower (C++, pure pursuit)

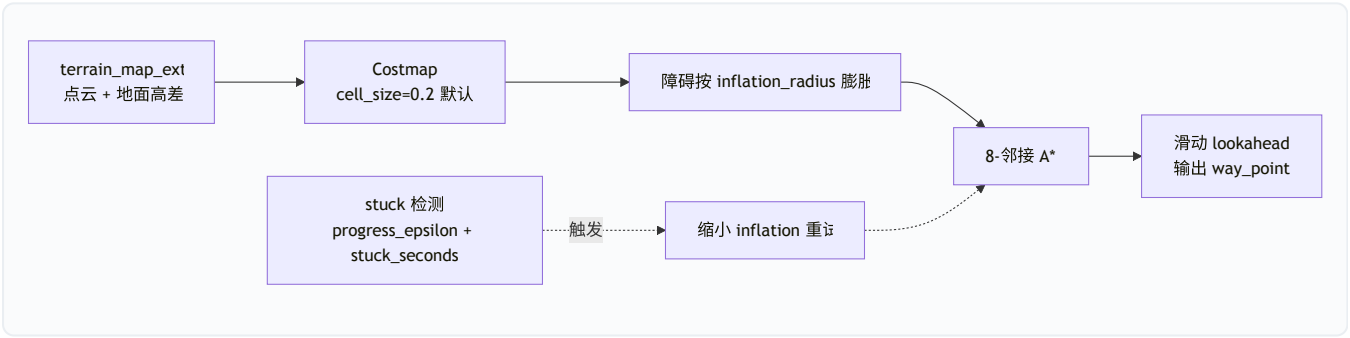
方向	端口	类型	说明
In	path	Path	来自 LocalPlanner
In	odometry	Odometry	corrected_odometry
In	speed	Float32	限速
In	slow_down	Int8	减速
In	safety_stop	Int8	紧急停
Out	cmd_vel	Twist	在 main.py 里 remap 成 nav_cmd_vel

关键配置：

字段	默认	含义
<code>look_ahead_distance</code>	0.5	pure pursuit lookahead 距离 (m)
<code>max_speed</code>	1.0	最大线速度 (m/s)
<code>max_yaw_rate</code>	45.0	最大角速度 (deg/s, C++ 内部转 rad)
<code>goal_tolerance</code>	0.3	终点容差 (m)
<code>vehicle_config</code>	<code>omniDir</code>	全向 / 标准
<code>max_acceleration</code>	1.0	最大加速度

7.5.6 SimplePlanner — Python 版的简化全局规划

如果你不想编译 FarPlanner 的 C++ 二进制，可以用 `planner="simple"`：



特点：

- 完全 Python 实现，调试方便
- 内置卡住时缩小障碍膨胀的容错逻辑（`stuck_shrink_factor`）
- 适合测试/开发，性能上不如 FarPlanner

7.5.7 NavRecord — 专用导航录制

把以下 14 个 topic 全录到 SQLite（默认 `nav_recording.db`）：

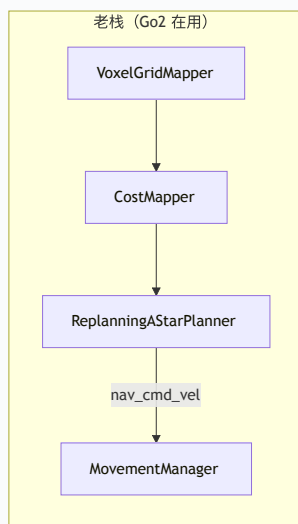
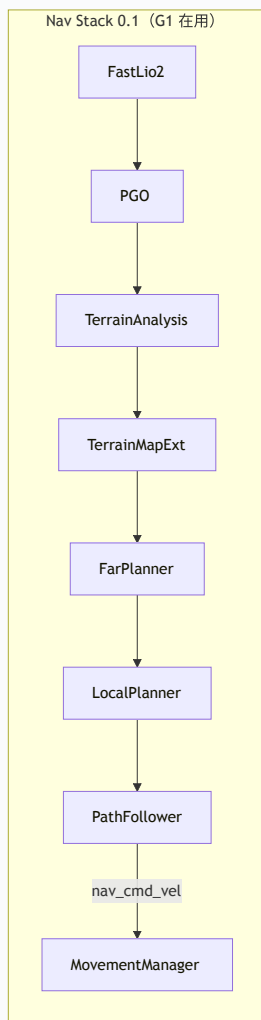
```
cmd_vel, corrected_odometry, path, goal_path,
way_point, goal, stop_movement, effective_cmd_vel,
slow_down, goal_reached, terrain_map, global_map,
odometry, registered_scan
```

后续可以用 dimos 的 rosbag fixture 工具

（`dimos/navigation/nav_stack/tests/rosbag_fixtures.py`）回放这些数据做离线测试。

注意：注释明确写了 G1 onboard 上启用 `record=True` 可能触发 TLS 内存分配失败（aarch64 + Python 多进程的已知问题）。所以 `main.py` 用的是 lazy import，`record=False` 时根本不导入。

7.5.8 老栈 vs 新栈 一图懂



两个栈通过 `nav_cmd_vel` 接口对齐 `MovementManager`——所以你完全可以"半新半老"混用，这也是为什么 commit `2a430b55b` 没把老栈删掉。

7.6 快速 cheatsheet

老栈（Go2 用）

想做的事	看哪
改 voxel 大小	<code>VoxelGridMapper.config.voxel_size</code> （默认 0.05）
改机器人尺寸	<code>GlobalConfig.robot_width</code> / <code>robot_rotation_diameter</code>
改 A* unknown 区域成本	<code>min_cost_astar(unknown_penalty=...)</code> （默认 0.8）
改控制频率	<code>LocalPlanner._control_frequency</code> （默认 10Hz）
改导航最大速度	<code>LocalPlanner._speed</code> （默认 0.55 m/s）
改卡住判定	<code>GlobalPlanner._stuck_time_window=8s</code> , <code>_stuck_threshold=0.4m</code>
改重规划阈值	<code>GlobalPlanner._max_path_deviation=0.9m</code>
调试模式画导航 cost map	<code>export DEBUG_NAVIGATION=1</code>
调试 frontier 评分	看 logs, <code>logger.info("Distance score: ...")</code>

Nav Stack 0.1（G1 用）

想做的事	看哪
一行起栈	<code>create_nav_stack(planner="far"/"simple", ...)</code>
切换全局规划器	<code>planner="far"</code> （C++ visibility graph） / <code>"simple"</code> （Python A*）
改机器人身高	<code>vehicle_height=1.5</code> （影响 TerrainAnalysis、TerrainMapExt、FarPlanner）
改最大速度	<code>max_speed=1.0</code> （同时传给 LocalPlanner + PathFollower）
改 lookahead	<code>path_follower={"look_ahead_distance": 0.5}</code>
启用录制	<code>record=True</code> （默认关，G1 onboard 慎开）
启用探索	<code>use_tare=True</code>
改 PGO 回环搜索半径	<code>pgo={"loop_search_radius": 1.0}</code>
改地形格大小	<code>terrain_voxel_size=0.2</code>
加载 Rerun 默认配色	<code>nav_stack_rerun_config(agentic_debug=True)</code>

FastLIO2（2026-05 新增）

想做的事	看哪
设置传感器安装位姿	<code>FastLio2.config.mount = Pose(x=..., y=..., z=..., orientation=...)</code>
输出 frame 名	<code>frame_id="odom"</code> （之前默认是 <code>"map"</code> ，新版用 <code>"odom"</code> ）
子 frame 名	<code>child_frame_id="body"</code>

文档基于 [d2e695b38](#)（2026-05-13 同步）。**Nav Stack 0.1（commit [2a430b55b](#)）已合并到 main，但 Go2 没换栈**——所以本教程的主体仍然有效，第七章新增 Nav Stack 0.1 章节作为 G1 / 自定义平台的参考。后续再有大变动会出新版。